

NutriMatch

Architecture applicative, DFD, STRIDE et mesures de securite

Application de recommandation nutritionnelle securisee

Etudiant 1	Benamara Marina Ferial – Specialite : ING4-SECI
Etudiant 2	Salah Oussam – Specialite : ING4-SECI

1 Objectif et perimetre

NutriMatch genere des recommandations de repas personnalisees selon le profil nutritionnel de l'utilisateur. Le systeme prend en compte les aliments aimes, les aliments moins apprecies, les allergies, les intolerances, les ingredients exclus, les maladies fermees et les medicaments declares.

La priorite du projet est la securite. Une recette dangereuse ne doit jamais etre proposee si elle viole une contrainte dure. Le backend reste donc l'autorite absolue : l'IA n'ajoute pas de recette externe, ne contourne pas les regles medicales et ne peut valider qu'une recette deja filteree par le moteur deterministe.

2 Architecture et stack technique

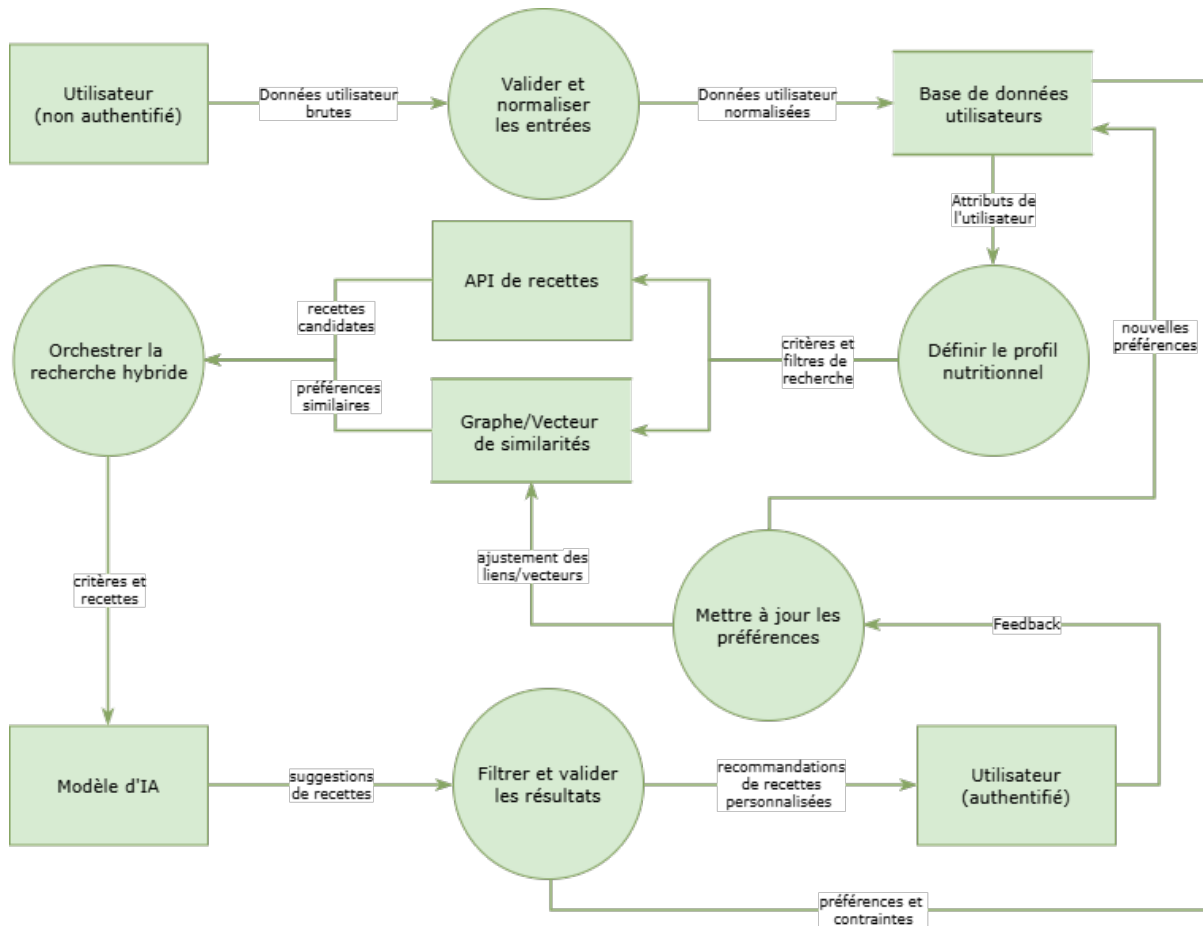
Composant	Role dans l'application
Frontend Next.js / React	Landing page, inscription, connexion, onboarding, resultats, choix de recette, profil et securite compte. Les pages protegees redirigent vers la connexion si la session est absente.
Backend Go / Gin	API REST, clean architecture, validation stricte, authentication, MFA, profil nutritionnel, moteur de recommandation, audit et orchestration IA.
PostgreSQL + pgvector	Persistence par schemas : identite, sante, catalogue local, recommandations, traces, audit et similarite vectorielle deterministe.
Redis	Support partage pour rate limiting, session/cache applicatif et reduction de charge.
Google AI optionnel	Validation et explication de recettes deja considerees sures par le backend. Les erreurs IA sont classees et exposees sans fuite de secret.
Docker Compose	Stack locale complete : base pgvector, Redis, migrations Goose, API compilee et frontend compile.

Le backend suit une clean architecture : handlers HTTP, middlewares, services metier, repositories et modeles. Les controles de securite ne sont pas delegues au frontend : chaque payload est revalide cote serveur.

3 Fonctionnement metier

1. L'utilisateur cree un compte, obtient une session puis complete son profil nutritionnel.
2. Le backend normalise les entrees, refuse les champs inconnus et calcule le profil nutritionnel utile.
3. Le catalogue local complet est charge depuis PostgreSQL ; aucune API recette externe n'est dans le chemin critique.
4. Les hard constraints bloquent sans negociation : allergies, allergies croisees, ingredients exclus, maladies, medicaments et regles nutritionnelles.
5. Les soft constraints influencent seulement le score : aliments aimes, aliments moins apprecies, objectif, similarite et diversite.
6. Le backend tire jusqu'a vingt recettes dans le pool deja sur, puis demande a l'IA de confirmer et expliquer ces recettes.
7. Si l'IA rejette une recette, le backend la retire pour la fenetre courante, tire une autre recette deja sure et relance la validation.
8. Le set quotidien est conserve vingt-quatre heures. Une recette choisie est affichee seule et exclue des futures suggestions pendant sept jours.

4 DFD logique et flux de donnees



- **Utilisateur vers frontend** : saisie du compte, profil, preferences et contraintes.
- **Frontend vers API** : requetes authentifiees, token CSRF pour mutations et access token court en memoire.
- **API vers PostgreSQL** : ecriture cloisonnee dans les schemas identite, sante, catalogue, recommandation et securite.
- **API vers Redis** : rate limit, session/cache et coordination legere.
- **API vers IA** : seulement des faits minimisees sur des recettes deja filtreees ; aucune donnee brute de sante inutile.
- **API vers frontend** : recettes validees, raisons metier, et explications IA seulement si la validation IA a reussi.

5 Modeles, donnees et cloisonnement

Schema	Contenu principal
identity	Utilisateurs, credentials, sessions, refresh tokens, MFA TOTP, passkeys WebAuthn et challenges.
health	Profil, mensurations utiles, preferences, contraintes et donnees sensibles chiffrées.
catalog	Recettes locales, ingredients, alias, allergies, allergies croisees, maladies et compatibilites medicales.
recommendation	Sets quotidiens, repas du set, choix utilisateur, rejets IA, details de preparation et traces.
security	Audit trail, hash-chain, evenements de securite, rate buckets et retention.

Le role applicatif PostgreSQL est separe du super-utilisateur local. Les donnees sensibles sont chiffrées au niveau applicatif avant persistance lorsque necessaire, et les index aveugles permettent certaines comparaisons sans exposer la valeur brute.

6 STRIDE par composant

6.1 1. Frontend Next.js et navigateur

S	Session volée ou page protégée ouverte sans session.	Guard via proxy Next.js, redirection vers <code>/login</code> , refresh cookie HttpOnly et access token uniquement en mémoire.
T	Manipulation du formulaire ou valeurs canoniques forcées.	Le frontend n'est pas fiable : validation finale côté API, taxonomie serveur et refus des champs inconnus.
R	Action utilisateur non tracable.	Les mutations sensibles génèrent des événements d'audit côté backend avec request ID.
I	XSS, fuite DOM, stockage local de données sante.	CSP avec nonce, pas de stockage persistant du profil sante dans le navigateur, cookies HttpOnly.
D	Abus de formulaires auth/profil.	Rate limiting côté API, body limit et validations par étape.
E	Navigation directe vers pages protégées.	Middleware/proxy de session et vérification serveur sur chaque endpoint protégé.

6.2 2. API Go, routes et middlewares

S	Requête anonyme se faisant passer pour un utilisateur.	JWT signé, expiration courte, session liée au token et vérification de session active.
T	JSON malformé ou injection de champs non prévus.	Binding JSON strict, DTO limitées, normalisation serveur et validation go-playground/validator.
R	Absence de preuve après incident.	Audit trail append-only, corrélation request/session/user et résultat de l'action.
I	Fuite par erreur API ou logs.	Reponses d'erreur standardisées, secrets jamais renvoyés, détails fournisseur IA non exposés.
D	Payloads volumineux ou flood endpoints.	Limite de body, rate limit Redis/PostgreSQL et timeouts externes.
E	BOLA/IDOR sur <code>profileId</code> ou recommandations.	Contrôle propriétaire de ressource dans les services et routes authentifiées.

6.3 3. Authentification, sessions, CSRF et MFA

S	Vol de credentials ou refresh token.	Mot de passe Argon2id, refresh token en cookie HttpOnly, hash du refresh avec pepper, rotation refresh et revocation logout.
T	CSRF sur refresh/logout/profil.	Token CSRF lié à la session, double submit cookie, header obligatoire, Origin/Referer de confiance.
R	Déni de changement mot de passe ou MFA.	Audit des changements password, MFA, passkeys, sessions et échecs de connexion.
I	Lecture du refresh token par JavaScript.	Cookie HttpOnly, SameSite configurable, Secure obligatoire en production.
D	Bruteforce login ou MFA.	Rate limiter, suivi des échecs auth, blocage temporaire et TTL de challenges MFA.
E	Accès fort sans second facteur configuré.	MFA optionnel mais complet : TOTP, passkeys WebAuthn, préférence MFA et challenge uniquement si active.

6.4 4. Profil sante et moteur nutritionnel

S	Lecture du profil d'un autre utilisateur.	User ID issu du token, ownership check et endpoints profil authentifies.
T	Contrainte medicale modifiee ou incoherente.	Taxonomie fermee pour maladies, ingredients canoniques, normalisation, validation et recalcul serveur.
R	L'utilisateur nie une contrainte renseignee.	Audit des modifications de profil, contraintes et recalculs nutritionnels.
I	Donnees sante exposees au repos.	Chiffrement AES-256-GCM cible, index aveugles, minimisation des champs et prompt IA reduit.
D	Profil trop verbeux ou trop complexe.	Budgets de texte, limites de listes et rejet des payloads excessifs.
E	Soft constraint transformee en bypass sante.	Hierarchie stricte : hard constraints avant soft scoring, similarite et IA.

6.5 5. Catalogue local, recommandations et IA

S	Source recette externe non controlee.	Catalogue local primaire en base ; suppression de l'API recette externe du chemin critique.
T	IA modifie l'ordre, ajoute une recette ou ignore une allergie.	Contrat JSON strict, IDs autorises uniquement, champs interdits refuses, revalidation backend apres IA.
R	Impossible d'expliquer une recommandation.	Traces de set quotidien, raisons metier, rejets IA, selection mode et decision summary.
I	Prompt contenant donnees sensibles inutiles.	Payload IA minimise : recettes deja sures, ingredients, contraintes abstraites et pas d'identifiant direct.
D	Recalcul couteux a chaque affichage.	Cache quotidien vingt-quatre heures, choix cache et exclusion sept jours.
E	Recette dangereuse acceptee par hasard ou similarite.	Pare-feu deterministe : allergies, allergies croisees, exclusions, maladies, medicaments et compatibilite medicale.

6.6 6. PostgreSQL, Redis et Docker

S	Connexion BDD avec droits trop larges.	Role applicatif separe du compte admin local et variables d'environnement dediees.
T	Alteration de traces ou donnees catalogue.	Migrations versionnees, audit hash-chain, repository pattern et transactions controlees.
R	Perte d'historique apres incident.	Audit immutable logique, retention configuree et traces securite separees.
I	Exposition de ports ou secrets Docker.	Ports lies a 127.0.0.1, reseaux internes Compose, secrets via env, pas de secrets en dur applicatifs.
D	Saturation DB/cache.	Healthchecks, Redis pour rate limit/cache, indexes, pgvector et timeouts.
E	Contournement par service interne.	Services separees, dependances Compose explicites, API demarre apres migrations et DB healthy.

7 Mesures de securite implementees

Domaine	Mesures concretes
Identite locale	Inscription/connexion, hachage Argon2id, politique longueur mot de passe, changement de mot de passe avec ancien mot de passe, confirmation et revocation des sessions pertinentes.
Refresh et sessions	Access token court en memoire, refresh token HttpOnly, hash refresh en base avec pepper, rotation au refresh, logout avec revocation, idle TTL et expiration.
MFA	TOTP complet, passkeys WebAuthn, challenge MFA conditionnel, preference MFA, desactivation controlee et audit des evenements MFA.
CSRF et origine	CSRF lie a la session, cookie CSRF lisible pour header, verification cookie/header, controle Origin/Referer et origines de confiance configurables.
CORS et cookies	CORS restreint aux origines autorisees, credentials controles, SameSite configurable, Secure obligatoire en production et chemins cookies dedies.
Frontend security	CSP avec nonce, frame-ancestors 'none' , protections headers, pas de profil sante stocke cote navigateur, access token non persistant.
Validation entrees	JSON strict, refus des champs inconnus, limites de taille body, budgets texte, normalisation email/texte/ingredients et taxonomie fermee.
Controle d'accès	Ownership checks sur profil, nutrition, recommandations, trace et choix de repas ; routes sensibles authentifiees.
Chiffrement repos	AES-256-GCM pour donnees sensibles cibles, chiffrement des secrets MFA et index aveugles avec cle separee.
Audit trail	Evenements auth, CSRF, MFA, profil, recommandations, choix, rejets, erreurs IA ; hash-chain, append-only logique et request/session correlation.
Retention	Parametres de retention pour audit, echecs auth, rate buckets, sessions et traces de recommandation.
Rate limiting	Backend compatible Redis ou PostgreSQL, buckets dedies, limitation auth et routes HTTP pour reduire brute force et DoS.
Catalogue local	Import nettoye, trim/dropna/deduplication, ingredients canoniques, allergies, allergies croisees et compatibilite maladie-recette.
Pare-feu nutritionnel	Blocage deterministe des allergies, intolerances, allergies croisees, ingredients exclus, maladies, medicaments et plafonds nutritionnels critiques.
Soft constraints	Likes, dislikes, objectif et similarite ne font que ponderer ; elles ne peuvent jamais annuler une contrainte dure.
IA controlee	Pool deja safe, contrat JSON strict, refus des IDs inconnus, refus des champs interdits, pas de score IA, pas de mock explanation, erreurs IA classees.
Cache recommandations	Set quotidien vingt-quatre heures, exclusions du set precedent si possible, exclusion sept jours apres choix et cache du detail IA choisi.
Docker	Images versionnees, PostgreSQL pgvector, Redis, migrations Goose automatiques, ports locaux standards, reseaux internes et healthchecks.

8 Paradigmes de securite

Zero Trust. Le navigateur, le reseau, les donnees entrantes et l'IA sont consideres non fiables. Toutes les decisions critiques sont reprises cote backend.

Defense in Depth. Le meme flux est protege par plusieurs couches : CSP, CORS, CSRF, JWT, session active, validation, chiffrement, audit, rate limit et controle metier.

Fail-Safe Defaults. En cas d'erreur medicale, de sortie IA invalide ou de doute sur une recette, la recette est rejetee ou l'explication IA est absente. Aucune contrainte dure n'est relachee.

Least Privilege. Separation des schemas, role applicatif dedie, prompt IA minimal, ports Docker limites localement et services separes.

Privacy by Design. Les donnees inutiles sont retirees, les donnees sensibles sont chiffrees, les prompts sont minimises et les logs evitent les secrets.

9 Limites assumees et conclusion

La version locale fonctionne en HTTP pour faciliter le developpement. Pour une production reelle, il faut imposer HTTPS, activer `COOKIE_SECURE=true`, utiliser des secrets forts, restreindre les origines HTTPS, proteger les logs, externaliser la supervision et appliquer TLS vers toute base distante.

NutriMatch est donc structure comme une application de recommandation nutritionnelle securisee : catalogue local, backend autoritaire, hard constraints non negociables, IA controlee et auditabilite complete des decisions sensibles.